

Backlog del Sprint 3 — Scaps

Carrito

Sprint 3 · 27 de julio al 9 de agosto de 2026. Con el catálogo, la ficha y la sesión funcionando desde los sprints anteriores, este sprint cierra el recorrido del MVP: el carrito, de punta a punta. En la API, los cuatro endpoints del carrito que define el contrato; en el frontend, el store con el contador del navbar, el selector de cantidad, el bloque de compra de la ficha, la pantalla del carrito y el cierre del pedido, más el botón para agregar desde las cards del catálogo y un cartel de últimas unidades. No hay migraciones: las tablas del carrito existen desde el Sprint 1. Al terminar tiene que estar desplegado: cualquiera puede crear una cuenta, agregar gorras al carrito, ajustar las cantidades y llegar al resumen del pedido.

Cada tarea indica su área entre corchetes, sus labels, qué hacer, el criterio para darla por terminada y, cuando aplica, de qué depende. La forma de trabajo (columnas del tablero, flujo y reglas) está en `Sprint-0-Scaps.pdf` y no se repite acá.

Antes de arrancar: el número de stock no se le muestra al cliente. Es información interna: nadie ve “Stock: 14” ni “quedan 3”, ni en la ficha (la guía visual lo muestra, pero no va), ni en el carrito, ni en un mensaje de error. Lo que sí se muestra son dos estados:

- **Sin stock.** En la card del catálogo ya está hecho: la card se apaga en tonos grises y lleva el cartel **Sin stock**. En la ficha, el cartel va en el bloque de compra, con el botón deshabilitado y sin selector de cantidad; la ficha se sigue pudiendo recorrer y el 3D se mira igual.
- **Pocas unidades.** Un cartel **Últimas unidades** o **Última unidad**, sin número. Lo define la tarea del cartel.

El campo `stock` sí viene en las respuestas de la API y se usa, pero solo para decidir comportamiento: el selector de cantidad topea sin mostrar el número (al llegar al máximo, el + se deshabilita y aparece “No hay más unidades disponibles”), y los mensajes de conflicto son cualitativos, como “No hay unidades suficientes de este producto, ajustá la cantidad”, nunca “quedan 2”.

El recorrido del MVP termina en el carrito. Quedan afuera el checkout real, el formulario de envío, Mercado Pago, `POST /orders` y todo lo de órdenes; la libreta de direcciones; el descuento automático de stock, porque el stock lo edita el admin y no se mueve solo; y el dashboard administrativo, que es el Sprint 4. Las tablas y los endpoints de órdenes y direcciones quedan documentados y sin usar en el contrato y el ERD: no se borran ni se implementan. Una consecuencia: **nada puede afirmar que se concretó una compra**. No hay orden en la base, así que una pantalla que diga “compra confirmada” o “gracias por tu compra” sería falsa. Cómo cierra el recorrido lo define la tarea del cierre del pedido.

Y reglas para todas las tareas:

- **Código en inglés:** componentes, variables, constantes y funciones. Los campos de la API van en español porque los fija el contrato (`precio` , `cantidad` , `subtotal`), y las rutas de la app también (`/carrito` , `/pedido`).
- **Stack mínimo:** ninguna tarea suma librerías. Todo sale con lo que ya está: React, Zustand, Tailwind, React Router, NestJS y Prisma.

- **Reutilizar, no rehacer:** el cliente HTTP (adjunta el token, traduce los errores de NestJS y avisa “despertando el servidor” cuando Render está dormido), `formatPrice`, `<ProductImage>`, el navbar, el store de sesión y el envoltorio de rutas protegidas ya existen. Se reutilizan y, donde una tarea lo pide, se completan, sin rehacerlos.
- **Responsive al 100 %:** cada pantalla se prueba con el modo dispositivo de las DevTools en cuatro anchos —360, 390, 768 y 1440 px— y en ninguno puede haber scroll horizontal ni un elemento cortado. Es parte del “hecho cuando”, no un pulido para después. Nada de anchos fijos en píxeles: las medidas de la guía son máximos de escritorio.
- **Estética:** tema oscuro, con los títulos en serif. El carrito sigue la línea del catálogo y la ficha: tiene que parecer parte de la misma app, no una pantalla pegada. La guía visual es una referencia de estética, no la especificación: **si una tarea y la guía no coinciden, vale la tarea.**
- **Errores y estados de carga en toda la interfaz:** ninguna pantalla se rompe ante una respuesta lenta, un `401` o un dato faltante.
- **Plata:** los montos viajan como string decimal (`"15999.00"`) y **el frontend nunca hace cuentas con plata:** los subtotales y el total los calcula el backend, y el front solo los muestra con `formatPrice`. Convertir el string a número para mostrarlo está bien; sumar, no.

To Do — se pueden tomar ya (no dependen de nada)

[carrito] Módulo de carrito en la API

carrito prioridad-alta

- **Qué hacer:** implementar los cuatro endpoints del carrito tal cual los define la sección 7 del contrato, en un módulo nuevo armado como el de productos (controller, service y DTOs). Los cuatro son autenticados: el guard de JWT ya deja disponible el usuario del token.
 - `GET /cart`: devuelve mi carrito. Si el usuario todavía no tiene uno, lo crea vacío en el momento y lo devuelve (nunca `404`). Vacío es `items: []` con `total: "0.00"`.
 - `POST /cart/items`: recibe `{ producto_id, cantidad }`. Si el producto ya está en el carrito, **suma** a la cantidad que había (el par `carrito_id + producto_id` es único en la base).
 - `PATCH /cart/items/:id`: recibe `{ cantidad }` y **reemplaza** la cantidad del ítem, no suma.
 - `DELETE /cart/items/:id`: quita el ítem.
- **Los tres últimos devuelven el carrito completo actualizado**, con la misma forma que el `GET`, así el frontend nunca recompone el estado a mano. Por eso los tres responden `200`: el `POST` lo declara con `@HttpCode(200)`, porque NestJS responde `201` por defecto en un POST, y el `DELETE` devuelve el carrito, no un `204` como el de productos.
- **Puntos donde se rompe si no se tiene cuidado:**
 - **El precio no se guarda en el ítem.** Se lee del producto en cada respuesta, y el backend calcula el `subtotal` de cada línea y el `total`. Las cuentas van con el `Decimal` de Prisma y se serializan con `toFixed(2)`, así los montos salen siempre con dos decimales, como en el contrato (`"31998.00"`). Nunca pasar por `float` en el medio.
 - **Stock:** si la cantidad resultante supera el `stock` del producto, `409`, en el `POST` y en el `PATCH`. El mensaje no lleva el número (ver “Antes de arrancar”): algo como “No hay unidades suficientes de Gorra Vanarsdale”, porque el frontend lo muestra tal cual.
 - **Pertenencia del ítem:** el `PATCH` y el `DELETE` reciben el id de un ítem. Antes de tocarlo, verificar que pertenece al carrito **del usuario del token**; si no, `404` (no `403`: no se le confirma a nadie

que ese id existe). Sin esta verificación, cualquier usuario logueado puede modificar el carrito de otro.

- **Productos inexistentes o dados de baja:** no se pueden agregar (404). Y si un producto se da de baja mientras está en el carrito de alguien, `GET /cart` lo omite y calcula el total sin él.
- **Validación de entrada:** `cantidad` es un entero ≥ 1 . Un `0` no es “borrar”, es `400`: para quitar está el `DELETE`. Los ids que no son uuid también son `400`: `ParseUUIDPipe` en la ruta, como en los endpoints de productos, y `@IsUUID` para el `producto_id` del body.
- **Crear con `upsert`, no con “buscar y, si no existe, crear”.** Vale para el carrito (uno por usuario), en cualquier endpoint que lo necesite, y para el ítem (uno por carrito y producto). En desarrollo, React ejecuta los efectos dos veces, y dos `GET /cart` simultáneos de un usuario nuevo chocarían contra la restricción de unicidad con un `500`.
- **Un orden fijo para los ítems.** Sin `orderBy`, Postgres los devuelve en cualquier orden, y una fila recién actualizada suele pasar al final: cada vez que alguien cambie una cantidad, esa línea saltaría al fondo del carrito. Ordenarlos siempre igual, por ejemplo por el nombre del producto.
- **actualizado_en del carrito** marca la última actividad (ver el modelo de datos), así que se actualiza con cada cambio de sus ítems. `@updatedAt` solo se mueve cuando cambia la fila del carrito, no las de sus ítems: hay que tocarlo a mano.
- **La imagen de cada producto:** `imagen_principal` tiene la misma forma que en la card del catálogo, y el módulo de productos ya la arma. Reutilizar esa lógica en vez de escribirla de nuevo.
- **Hecho cuando:** probados con un cliente de API y los tokens de dos usuarios distintos, los cuatro endpoints responden las formas del contrato; un usuario nuevo recibe un carrito vacío en su primer `GET`; agregar dos veces el mismo producto suma en vez de duplicar la fila; pasarse del stock devuelve `409` en el `POST` y en el `PATCH`, sin el número en el mensaje; un usuario no puede modificar ni borrar el ítem de otro; dar de baja un producto que está en un carrito no rompe el `GET`; cambiar la cantidad de un ítem no le cambia el lugar en la lista; y los totales cierran con dos decimales exactos.

Por qué va primero: el frontend del carrito no se puede probar de verdad sin la API. Mientras tanto, el selector de cantidad y la vuelta al origen después de ingresar no dependen de nada y se pueden tomar en paralelo.

[carrito] Componente del selector de cantidad

carrito prioridad-alta

- **Qué hacer:** un componente `<QuantitySelector>` (– / número / +) para elegir cuántas unidades. Es una pieza suelta, sin API ni store: recibe el valor, el tope y qué hacer con cada cambio. Lo usan el bloque de compra de la ficha y la pantalla del carrito, cada una con su propio tope, y va en una tarea propia para que las dos lo encuentren hecho y ninguna arme su versión.
- **Reglas:**
 - El mínimo es 1 y el tope llega por prop. Al llegar al tope, el + se deshabilita y aparece “No hay más unidades disponibles”, sin números (ver “Antes de arrancar”).
 - El número del medio se puede escribir, pero se valida al salir del campo o con Enter, no tecla por tecla: en el carrito, cada cambio es un pedido a la API. Vacío o letras vuelven al valor anterior; un número menor que 1, a 1; y uno mayor al tope, al tope.
 - **Si el valor que recibe ya supera el tope** (en el carrito pasa cuando el admin baja el stock de algo que alguien ya tenía), muestra el valor real y no lo corrige solo: el + queda deshabilitado, sin el aviso de tope (la pantalla muestra el suyo), y el – baja directo al tope, en un solo cambio. Restar de a uno no sirve: la API rechaza con `409` cualquier cantidad que siga pasándose, la línea no cambia y el – nunca la saca de ahí.

- Se puede deshabilitar entero, para cuando la pantalla tiene un pedido en curso.
- No se usa con un tope de 0: para ese caso, la ficha y el carrito tienen su propio estado.
- **Accesible y cómodo en el celular:** los botones llevan `aria-label` ("Restar una unidad", "Sumar una unidad"); el campo tiene su etiqueta e `inputMode="numeric"`, que en el celular abre el teclado numérico; y los controles mantienen la altura de la guía (44 px), que además es cómoda para el dedo.
- **Hecho cuando:** montado de prueba en cualquier página (y sacado antes de abrir el Pull Request), respeta el mínimo y el tope; al llegar al tope deshabilita el + y muestra el aviso; escribir 0 lo lleva a 1, un número mayor al tope lo lleva al tope, y vacío o letras vuelven al valor anterior; escribir no dispara ningún cambio hasta salir del campo o apretar Enter; con un valor por encima del tope, no muestra el aviso de tope y el - lo lleva al tope de una; se maneja entero con el teclado; y en los cuatro anchos de prueba se ve completo.

[auth] Volver a donde estaba al ingresar o crear la cuenta

auth prioridad-alta

- **Qué hacer:** dejar listo el regreso a la página de origen después de iniciar sesión o de registrarse, que van a usar los botones Agregar al carrito de la ficha y de las cards del catálogo. Una parte existe desde el Sprint 1: el envoltorio de rutas protegidas manda al login y, al iniciar sesión, la app vuelve a la ruta original (quien entra a `/carrito` sin sesión, después de ingresar está en el carrito, no en la landing). La tarea es verificar cómo está hecho y completarlo con ese mismo mecanismo, sin reescribirlo:
 - **Primero, ver qué guarda y dónde:** el `state` de la navegación o un parámetro de la URL. Todo lo que sigue usa eso mismo.
 - **Desde cualquier página, no solo desde una ruta protegida.** La ficha y el catálogo son públicos y van a mandar al login por su cuenta. Si hoy el envoltorio arma ese envío a mano, sacarlo a una función chica que pueda usar cualquier pantalla, así la forma de pasar el origen queda escrita en un solo lugar.
 - **Con sus parámetros.** El origen es la ruta completa, con lo que va después del `?`: los filtros del catálogo viven en la URL (`/catalogo?q=trucker&sort=precio_asc`), y la persona tiene que volver con los mismos. En React Router, `pathname` más `search`. Si el origen viaja en un parámetro de la URL, va codificado con `encodeURIComponent`: si no, el `&` de los filtros lo corta. Y al volver, solo se aceptan rutas internas (que empiezan con una sola `/`): si no, cualquiera podría armar un enlace al login que, después de ingresar, lleve a otro sitio.
 - **También al crear la cuenta.** Si la persona pasa del login a **Crear cuenta**, el origen viaja con ella y, al registrarse, vuelve ahí. Lo mismo al revés, del registro a **Ingresar**.
 - **Sin origen, todo sigue como hoy:** Ingresar o Crear cuenta desde el navbar llevan a la landing al terminar.
- **Hecho cuando:** entrar a `/carrito` sin sesión lleva al login y, al ingresar, vuelve a `/carrito`; el mismo recorrido pasando del login a Crear cuenta y registrándose también vuelve a `/carrito`; con parámetros en la URL (por ejemplo, entrando a `/carrito?prueba=1`) vuelve con ellos; si el origen viaja en la URL, uno que apunta a otro sitio lleva a la landing; hay una sola forma de mandar al login con el origen, lista para que la usen la ficha y el catálogo; e ingresar o registrarse desde el navbar sigue llevando a la landing.

Backlog — esperan que se libere una dependencia

[carrito] Store del carrito y contador en el navbar

carrito prioridad-alta

- **Depende de:** Módulo de carrito en la API.
- **Qué hacer:** un store de Zustand para el carrito, al lado del de sesión, que sea la única fuente de verdad del carrito en el frontend: guarda el carrito que devolvió la API y expone las acciones para cargarlo, agregar, cambiar una cantidad y quitar. Como cada endpoint devuelve el carrito completo, cada acción reemplaza el estado con la respuesta: no se recalcula nada del lado del cliente.
- **Además del carrito, el estado de la carga:** si todavía no llegó, si está listo o si falló. Las pantallas lo necesitan para distinguir “todavía no cargó” de “está vacío”: sin eso, al recargar `/carrito` se vería un instante “Tu carrito está vacío”.
- **Cuándo carga y cuándo se vacía:**
 - **Solo con sesión.** Sin sesión no se llama a `GET /cart`: devuelve `401` y ensucia la consola.
 - **Carga una vez cuando la sesión está confirmada:** al arrancar la app, después de que `GET /auth/me` confirma el token (no en paralelo: con un token vencido, el carrito también daría `401`), y al iniciar sesión o registrarse.
 - **Se vacía al cerrar sesión**, y también cuando la sesión vence y se descarta. Si no, el carrito del usuario anterior queda a la vista del siguiente.
 - Lo más simple es que el store del carrito se suscriba al de sesión (Zustand trae `subscribe`): cuando aparece un usuario, carga; cuando se va, se vacía. Así cubre todos los casos sin tocar el login, el registro ni el cierre de sesión uno por uno.
- **En el navbar, el ícono de un carrito**, sin texto, como en cualquier tienda. Lleva a `/carrito` y va solo con sesión: en escritorio, al final de la barra, después de Cerrar sesión; en el celular, siempre a la vista en la barra, al lado del botón del menú y no dentro del menú desplegable, así el número se ve sin abrir nada.
 - **El contador:** un globito en la esquina del ícono con la cantidad total de **unidades**, no de líneas: dos gorras iguales más una distinta son 3. Sin unidades se ve el ícono solo, sin globito, y desde 100 dice “99+”. Ese conteo lo expone el store, porque la pantalla del carrito también lo muestra.
 - **De dónde sale el ícono:** del mismo lugar que el cubo de **Ver en 3D**, así los dos se ven de la misma familia. Si el cubo viene de una librería que ya está instalada, se usa esa; si es un SVG, el carrito también. No se suma una librería por un ícono.
 - **Accesible:** como no tiene texto, el enlace lleva un `aria-label` con la cantidad, por ejemplo “Carrito, 3 unidades” (o “1 unidad”); sin eso, un lector de pantalla no anuncia nada. El área para tocarlo mide 44 px, aunque el ícono sea más chico. En `/carrito` se marca como sección activa, igual que los demás enlaces.
- **Hecho cuando:** el ícono aparece solo con sesión; el contador arranca con el número correcto al recargar la página y se actualiza en el momento cada vez que una acción del store cambia el carrito (mientras no existan las pantallas, se prueba con un botón de prueba que se saca antes de abrir el Pull Request); sin unidades, el ícono no muestra globito; el `aria-label` cambia con la cantidad; al cerrar sesión el ícono desaparece y, al ingresar con otro usuario, el contador es el de ese usuario, no el del anterior; sin sesión, la pestaña *Network* no muestra ningún pedido a `/cart`; y en los cuatro anchos de prueba el navbar se ve completo, con el ícono en la barra y, en el celular, fuera del menú.

[carrito] Bloque de compra en la ficha de producto

carrito prioridad-alta

- **Depende de:** Store del carrito y contador en el navbar + Componente del selector de cantidad + Volver a donde estaba al ingresar o crear la cuenta.
- **Qué hacer:** sumar a la ficha, debajo de la descripción, el bloque que falta: el `<QuantitySelector>` y el botón **Agregar al carrito**. El número de stock no se muestra (ver “Antes de arrancar”).
 - **El tope del selector es lo que todavía se puede agregar:** el stock menos las unidades de ese producto que ya están en el carrito, que el store tiene. El `POST` suma a lo que ya había: con el stock solo como tope, quien tiene 3 de 5 en el carrito podría elegir 5 y se comería un `409`, sin forma de saber cuántas le quedan. Sin sesión, el tope es el stock; con sesión, el botón espera a que el carrito haya cargado. Esa cuenta va en una sola función (por ejemplo, `getAvailableToAdd`), porque el botón de las cards del catálogo también la usa.
 - **Si ya están todas en el carrito** (el tope da 0 o menos y el producto tiene stock): el bloque muestra “Ya tenés en el carrito todas las unidades disponibles” con el enlace **Ver carrito**, sin selector y con el botón deshabilitado.
 - **Sin stock:** el cartel **Sin stock** en el bloque, el botón deshabilitado y sin selector. Si la ficha ya muestra un Sin stock entre los datos del producto (lo pedía el Sprint 2), pasa al bloque: queda uno solo.
 - **Agregar al carrito** llama al store y se deshabilita mientras el pedido está en curso, para que un doble clic no agregue dos veces. Al terminar, el selector vuelve a 1 y aparece una confirmación breve en la misma pantalla (por ejemplo, “Agregado al carrito”) con un enlace **Ver carrito**. No redirigir al carrito: la persona puede querer seguir mirando.
 - **Sin sesión,** el botón se ve igual, pero al tocarlo manda al login con la función de “Volver a donde estaba”, así que después de ingresar, o de crear la cuenta, la persona vuelve a esta ficha. No esconderlo ni deshabilitarlo: esconderlo le oculta al visitante que el producto se puede comprar.
 - **Si la API responde `409` o `404`** (entre que se cargó la ficha y el clic, el admin bajó el stock o dio de baja el producto), mostrar el aviso que vino del backend y volver a pedir la ficha, así la pantalla queda al día. Con el `404`, la ficha pasa a “Producto no encontrado”, como ya hace.
 - **Las cards no se tocan en esta tarea.** El botón de las cards del catálogo tiene su propia tarea, y las de “También te puede interesar” siguen con **Ver detalles**.
- **En el celular:** el bloque ocupa todo el ancho, con el selector y el botón en una fila, como en la guía, y el botón estirado hasta el borde.
- **Hecho cuando:** en ningún lado se ve el número de stock; agregar actualiza el contador del navbar; el doble clic agrega una sola vez; con un producto de 5 unidades (bajándole el stock desde `prisma studio`) y 3 de ellas en el carrito, el selector topea en 2, y con las 5 en el carrito aparece el aviso de que ya están todas, sin selector; un producto sin stock muestra el cartel y el botón deshabilitado; sin sesión, el botón lleva al login y después vuelve a la misma ficha, también si la persona se registra en vez de ingresar; bajándole el stock desde `prisma studio` con la ficha abierta, el `409` muestra el aviso y la ficha se actualiza; y en los cuatro anchos de prueba el bloque se ve completo, sin scroll horizontal.

[carrito] Pantalla del carrito

carrito prioridad-alta

- **Depende de:** Store del carrito y contador en el navbar + Componente del selector de cantidad.

- **Qué hacer:** construir la pantalla `/carrito` con el carrito del store. La ruta ya existe y está protegida desde el Sprint 1: sin sesión manda al login y, al ingresar, vuelve.
 - **Por cada ítem:** la imagen con `<ProductImage>` en un área cuadrada, como en el catálogo; el nombre, como enlace a su ficha; el precio unitario; el `<QuantitySelector>`, con el stock como tope; el subtotal de la línea; y un botón **Quitar** bien identificado, no un ícono suelto sin etiqueta accesible. Quitar puede ser directo, sin confirmación. Los montos, con `formatPrice`.
 - **El resumen**, en una columna al costado desde `lg` (1024 px), como en la guía, y debajo de los ítems en los anchos menores: la cantidad de unidades, la misma que el store le da al navbar, y el **total**, tal cual viene de la API. El botón Finalizar compra lo suma la tarea del cierre del pedido.
 - **Cada cambio va a la API:** cambiar una cantidad o quitar un ítem llama al store, que reemplaza el carrito con la respuesta. Mientras el pedido está en curso se deshabilitan los controles de esa línea, no la pantalla entera. Si la API responde `409` o `404` (por ejemplo, el ítem ya se quitó desde otra pestaña), mostrar el aviso y volver a cargar el carrito.
 - **Estados:** mientras carga, un indicador, nunca un “Tu carrito está vacío” que después desaparece (para eso el store expone si ya cargó); si falla, el error con un botón **Reintentar**; y vacío, “Tu carrito está vacío” con un botón **Ver catálogo**, no una pantalla en blanco.
- **Líneas que se pasan del stock:** si el admin bajó el stock de un producto después de que alguien lo agregó, la línea lo avisa de forma cualitativa: “No hay unidades suficientes de este producto, ajustá la cantidad”. El selector muestra la cantidad que hay, con el + deshabilitado y el – que baja directo al máximo (lo resuelve el componente). Si el producto se quedó sin stock, la línea muestra **Sin stock**, sin selector, y solo se puede quitar. La regla de “esta línea se pasa del stock” va en una sola función (por ejemplo, `exceedsStock`), porque la tarea del cierre del pedido la reutiliza. Los productos dados de baja no llegan: `GET /cart` ya los omite.
- **Lo que no va de la guía:** la línea de Envío, el SKU y el botón Continuar con el checkout. El envío y el checkout salieron del MVP.
- **En el celular:** cada ítem se apila (la imagen y el nombre arriba; el selector, el subtotal y Quitar abajo) y el resumen va al final, a todo el ancho. Nada de una tabla que obligue a scrollear de costado.
- **Hecho cuando:** se ven los ítems con sus subtotales y el total correcto; cambiar una cantidad actualiza el subtotal, el total y el contador del navbar, y la línea no cambia de lugar; quitar el último ítem deja el estado vacío con su botón; recargar la página muestra la carga y después el carrito, sin pasar por el vacío; con el backend apagado aparece el error con Reintentar; bajándole el stock a un producto del carrito desde `prisma studio` por debajo de su cantidad, la línea avisa, el + queda deshabilitado y el – la lleva al máximo con un solo pedido; con el stock en 0, la línea muestra Sin stock y solo se puede quitar; y en los cuatro anchos de prueba la pantalla se ve completa, sin scroll horizontal, con el resumen al costado en 1440 y debajo en los otros tres.

[carrito] Finalizar compra y cierre del pedido

carrito prioridad-alta

- **Depende de:** Pantalla del carrito.
- **Qué hacer:** sumar al resumen del carrito el botón primario **Finalizar compra** y construir la pantalla a la que lleva, `/pedido`, que cierra el recorrido del MVP. Es lo último que ve quien recorre la demo, así que tiene que quedar prolija.
 - **Finalizar compra** queda deshabilitado mientras alguna línea se pasa del stock (con la función de la pantalla del carrito), con el motivo a la vista, por ejemplo: “Ajustá las cantidades marcadas para continuar”.
 - `/pedido` es una ruta nueva, protegida como el carrito, encabezada por **“Tu pedido está listo”** y con una línea que aclara que Scaps es un proyecto demo y que el pago en línea llega en la

próxima etapa. Muestra el resumen de lo que se iba a comprar (cada ítem con su cantidad y su subtotal, y el total) y tiene dos salidas: **Volver al catálogo** y **Volver al carrito**.

- **El carrito no se vacía**, y la pantalla lo dice: el pedido queda guardado en la cuenta. Es cierto, porque el carrito se guarda por usuario, y es justamente lo que el MVP sí hace.
- **No afirma que se compró nada**: nada de “compra confirmada”, “gracias por tu compra” ni un número de pedido inventado (ver “Antes de arrancar”).
- **Solo con un carrito válido**. Si alguien entra a `/pedido` con el carrito vacío o con una línea que se pasa del stock (escribiendo la URL o volviendo con el botón Atrás), redirigirlo al carrito. Ojo con el orden: al recargar `/pedido`, esperar a que el carrito termine de cargar antes de decidir; en el primer instante el store todavía está vacío, y la recarga redirigiría siempre. Si la carga falla, el error con **Reintentar**, no la redirección.
- **En el celular**: el resumen va a todo el ancho, y las dos salidas, una debajo de la otra, también.
- **Hecho cuando: Finalizar compra** lleva a `/pedido` con el resumen correcto, y el carrito y el contador del navbar quedan intactos; con una línea que se pasa del stock, el botón está deshabilitado y se ve el motivo; `/pedido` con el carrito vacío redirige al carrito, y recargarlo con un carrito válido no; entrar a `/pedido` sin sesión manda al login y, al ingresar, vuelve; ninguna pantalla dice que se concretó una compra; y en los cuatro anchos de prueba se ve completa, sin scroll horizontal.

[carrito] Agregar al carrito desde el catálogo

carrito prioridad-media

- **Depende de**: Bloque de compra en la ficha de producto + Cargar más, del Sprint 2.
- **Qué hacer**: en las cards de la grilla del catálogo, cambiar el botón **Ver detalles** por **Agregar al carrito**, que suma una unidad sin salir del catálogo. Como las gorras no tienen talle ni color, no hay nada que elegir antes de agregar. Ver detalles deja de hacer falta: tocar la card sigue abriendo la ficha.
 - **Solo en el catálogo**. “También te puede interesar” usa la misma card, y ahí sigue Ver detalles: en la ficha, el único botón de compra es el del producto que se está mirando. La card recibe por prop cuál de los dos botones muestra.
 - **Cada clic suma una unidad** con la acción de agregar del store, y se puede seguir sumando hasta el tope. Mientras el pedido está en curso, el botón se deshabilita; al terminar, muestra “Agregado” y sigue deshabilitado un momento (un segundo y medio alcanza) antes de volver a “Agregar al carrito”. Así un doble clic no suma dos sin querer, y el contador del ícono del navbar confirma el cambio. Para llevar varias unidades de una vez están el selector de la ficha y el carrito.
 - **El mismo tope que la ficha**, con la función de unidades disponibles del bloque de compra; con sesión, el botón espera a que el carrito haya cargado. Si ya están en el carrito todas las unidades disponibles, el botón queda deshabilitado y dice “En tu carrito”. Sin stock, el botón queda deshabilitado; la card sigue gris, con su cartel, y se puede abrir igual.
 - **Sin sesión**, el botón se ve igual, y al tocarlo manda al login con la función de “Volver a donde estaba”: después de ingresar, o de crear la cuenta, la persona vuelve al catálogo con los mismos filtros.
 - **Si el pedido falla**, por ejemplo con un `409` o un `404` (entre que se cargó el catálogo y el clic, el admin bajó el stock o dio de baja el producto), el botón muestra “No se pudo agregar” un momento, igual que con “Agregado”, y el contador no cambia. No se recarga el catálogo, porque se perderían las tandas de Cargar más.
- **Un botón no puede ir adentro de un enlace**. Hoy la card entera es un `<a>`, y un botón adentro de un enlace es HTML inválido: el clic, además de agregar, abriría la ficha. La card se rearma así: el

enlace pasa a ser el nombre del producto, y un `::after` del enlace estirado sobre toda la card (`after:absolute after:inset-0`, con la card en `relative`) hace que se pueda tocar en cualquier parte, como hasta ahora. El botón va por encima (`relative z-10`), así su clic no llega al enlace. En las cards que siguen con Ver detalles, ese botón queda debajo del enlace estirado (sin `z-10`) y abre la ficha como el resto de la card.

- **En el celular**, la card no muestra el botón, igual que hoy no muestra Ver detalles (`hidden md:block`): a ese ancho le quita lugar al nombre y al precio, y la ficha está a un toque.
- **Hecho cuando:** en el catálogo, cada card muestra Agregar al carrito en lugar de Ver detalles, y en “También te puede interesar” sigue Ver detalles; cada clic suma una unidad y sube el contador del ícono; un doble clic rápido suma una sola; con un producto de 5 unidades (bajándole el stock desde `prisma studio`) y el carrito vacío, la card agrega hasta 5 y después el botón queda deshabilitado con “En tu carrito”; sin stock, el botón está deshabilitado y la card se sigue pudiendo abrir; tocar la card fuera del botón abre la ficha, y tocar el botón no; con el teclado se llega al enlace y al botón por separado; sin sesión, el botón lleva al login y después vuelve al catálogo con los mismos filtros, también si la persona crea la cuenta; en ningún lado se ve el número de stock; y en los cuatro anchos de prueba las cards de cada fila siguen parejas, sin el botón en el celular.

Por qué prioridad media: el recorrido no la necesita, porque la ficha ya tiene el botón; si el tiempo se corta, se mueve después del cartel. **Por qué espera a Cargar más:** ese botón del Sprint 2 también toca la grilla y seguía en curso al armar este backlog; si ya está en `main`, la dependencia está cumplida.

[catalogo] Cartel de últimas unidades

catalogo prioridad-media

- **Depende de:** Agregar al carrito desde el catálogo.
- **Qué hacer:** un cartel que avise cuando quedan pocas unidades de un producto, como hacen muchas tiendas. Nunca muestra el número (ver “Antes de arrancar”).
 - Con 2 a 5 unidades dice **Últimas unidades**; con 1, **Última unidad**. Con 0 sigue el tratamiento de **Sin stock** que ya existe: los dos carteles nunca conviven.
 - El umbral va en una constante, `LOW_STOCK_THRESHOLD = 5`, y la regla en una sola función que usan la card y la ficha, así no se escribe dos veces.
 - Va en la **card** (con eso aparece también en “También te puede interesar”, que usa la misma card) y en la **ficha**, junto al bloque de compra.
 - Siempre sale del `stock` real que manda la API, nunca de un valor fijo.
 - Se distingue a primera vista del cartel gris de **Sin stock**, sin competir con el precio y sin romper las cards parejas de la grilla: una card con cartel y una sin cartel, en la misma fila, quedan del mismo alto.
- **Para probarlo**, bajá el stock de un producto en tu base de desarrollo desde `prisma studio`. El seed y producción no se tocan: dejar un producto con pocas unidades para la demo se resuelve aparte.
- **Hecho cuando:** con 1 unidad se ve “Última unidad”; con 2 a 5, “Últimas unidades”; con 0, solo “Sin stock”; con más de 5, nada; el cartel aparece igual en la card del catálogo, en “También te puede interesar” y en la ficha; en ningún lado se ve el número; y en los cuatro anchos de prueba las cards de cada fila siguen parejas.

Por qué prioridad media: si el tiempo se corta, es la primera tarea que se mueve, y después Agregar al carrito desde el catálogo; ninguna de las dos rompe nada. **Por qué va después del botón del catálogo:** las dos tocan la card, y así no se pisan.